

Наив Байесын алгоритмаар спам и-мэйл илрүүлэх НЬ

4-р баг

2026 оны 5-р сарын 7

Энэхүү ажлаар спам болон хэвийн и-мэйлүүдийг машин сургалтын Наив Байесын алгоритмын тусламжтай ангилах загвар боловсруулав. Текст өгөгдлийг Bag-of-Words аргаар тоон хэлбэрт хөрвүүлж, Лапласын smoothing ашиглан магадлалын загвар сургаж, улмаар хөгжүүлэлтийн өгөгдөл дээр үр дүнг үнэлэв. Судалгааны үр дүнд боловсруулсан загвар 97.20 хувийн нарийвчлалтай ажилласан.

Агуулга

1 Оршил	2
1.1 Өгөгдөл	2
1.2 Судалгааны өгөгдлийн статистик шинжилгээ	2
1.2.1 Хураангуй статистик хүснэгт	3
1.2.2 Ангиллын тархалтын дүрслэл	4
2 Онолын үндэс	6
2.1 Байесын теорем	6
2.2 Загварын үндсэн таамаглал	7
3 Өгөгдлийн статистик боловсруулалт	7
3.1 Текст өгөгдлийн боловсруулалт	7
3.2 И-мэйлийн ургын статистик	8
3.3 Үгийн сан байгуулах	8
3.4 Bag-of-Words статистик дүрслэл	9
3.5 Урьдчилсан магадлалын үнэлгээ	11
3.6 Нөхцөлт магадлалын үнэлгээ	12
3.7 Загвар сургах	12
3.8 Confusion matrix	13
3.9 Accuracy	14
3.10 Precision	14
3.11 Recall	14
3.12 F1-score	15
3.13 Confusion matrix дүрслэл	16
4 Үр дүнгийн статистик тайлбар	17
5 Загварын хязгаарлалт	19
6 Дүгнэлт	19

1 Оршил

Цахим шуудангийн хэрэглээ нэмэгдэхийн зэрэгцээ хүсээгүй мэдээлэл буюу спам и-мэйлүүдийн хэмжээ өсөн нэмэгдэж байна. Ийм мэдээллийг автоматаар ангилах асуудал нь магадлалын онол, статистикийн аргачлалд тулгуурласан ангиллын асуудлын нэг юм. Энэхүү судалгааны ажлын зорилго нь Наив Байесын магадлалын загварыг ашиглан спам болон хэвийн и-мэйлийг ангилах статистик загвар боловсруулахад оршино. # Судалгааны өгөгдөл

1.1 Өгөгдөл

Судалгаанд спам болон хэвийн и-мэйлүүдээс бүрдэх текстэн өгөгдлийг ашиглав.

```
from src.dataset import read_dataset

train_emails, train_labels = read_dataset(
    "../data/train"
)

dev_emails, dev_labels = read_dataset(
    "../data/dev"
)

print(
    "Train:",
    len(train_emails)
)

print(
    "Dev:",
    len(dev_emails)
)
```

Train: 2000

Dev: 1000

1.2 Судалгааны өгөгдлийн статистик шинжилгээ

Судалгаанд ашигласан өгөгдлийн бүтэц, ангиллын тархалт, болон ажиглалтын тоон үзүүлэлтүүдийг эхлэн шинжилэв.

Нийт өгөгдлийг сургалтын болон хөгжүүлэлтийн өгөгдөлд хуваасан бөгөөд ангилал бүрийн ажиглалтын тоог тооцоолов.

```
train_spam = sum(  
    label == "spam"  
    for label in train_labels  
)  
  
train_ham = sum(  
    label == "ham"  
    for label in train_labels  
)  
  
dev_spam = sum(  
    label == "spam"  
    for label in dev_labels  
)  
  
dev_ham = sum(  
    label == "ham"  
    for label in dev_labels  
)  
  
print(  
    "Train spam:",  
    train_spam  
)  
  
print(  
    "Train ham:",  
    train_ham  
)  
  
print(  
    "Dev spam:",  
    dev_spam  
)  
  
print(  
    "Dev ham:",  
    dev_ham  
)
```

Train spam: 1000

Train ham: 1000

Dev spam: 500

Dev ham: 500

Дээрх үр дүнгээс өгөгдлийн ангиллын тархалт харьцангуй тэнцвэртэй байгааг ажиглаж болно.

1.2.1 Хураангуй статистик хүснэгт

Үзүүлэлт	Утга
Сургалтын өгөгдөл	<code>r len(train_emails)</code>
Хөгжүүлэлтийн өгөгдөл	<code>r len(dev_emails)</code>
Сургалтын spam	<code>r train_spam</code>
Сургалтын ham	<code>r train_ham</code>
Хөгжүүлэлтийн spam	<code>r dev_spam</code>
Хөгжүүлэлтийн ham	<code>r dev_ham</code>

1.2.2 Ангиллын тархалтын дүрслэл

```

train_total = (
    train_spam +
    train_ham
)

dev_total = (
    dev_spam +
    dev_ham
)

train_spam_ratio = (
    train_spam /
    train_total
)

train_ham_ratio = (
    train_ham /
    train_total
)

dev_spam_ratio = (
    dev_spam /
    dev_total
)

dev_ham_ratio = (
    dev_ham /
    dev_total
)

counts = [
    train_spam_ratio,
    train_ham_ratio,
    dev_spam_ratio,
    dev_ham_ratio
]

labels = [
    "Train spam",

```

```

    "Train ham",
    "Dev spam",
    "Dev ham"
]

plt.figure(
    figsize=(6,4)
)

plt.bar(
    labels,
    counts
)

plt.ylabel(
    "Relative frequency"
)

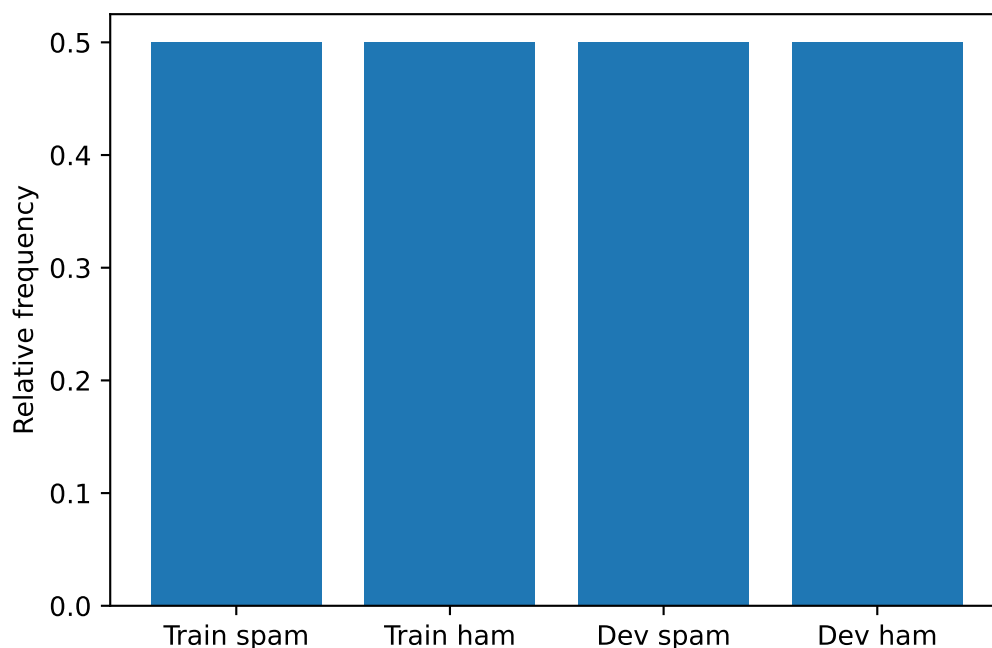
plt.show()

print(
    "Train spam ratio:",
    round(
        train_spam_ratio,
        3
    )
)

print(
    "Dev spam ratio:",
    round(
        dev_spam_ratio,
        3
    )
)

```

Figure 1: Ангиллын харьцангуй тархалтын дүрслэл



Train spam ratio: 0.5

Dev spam ratio: 0.5

Дээрх дүрслэлээс сургалтын болон хөгжүүлэлтийн өгөгдөл дэх ангиллын харьцангуй давтамжууд ойролцоо байгааг ажиглаж болно.

Энэ нь өгөгдлийн хуваарилалт статистикийн хувьд тогтвортой байгааг илэрхийлэх бөгөөд загварын үнэлгээний найдвартай байдалд эерэг нөлөө үзүүлнэ.

2 Онолын үндэс

2.1 Байесын теорем

Энэхүү судалгаанд текстэн өгөгдлийг ангилах статистик аргачлал болох Наив Байесын ангиллын загварыг ашиглав. Уг арга нь Байесын теоремд үндэслэн ажиглагдсан өгөгдөл өгөгдсөн нөхцөлд тухайн объект тодорхой ангилалд хамаарах магадлалыг тооцоолдог.

Байесын теоремийг дараах байдлаар илэрхийлнэ.

$$P(C | X) = \frac{P(X | C)P(C)}{P(X)}$$

Энд:

- $P(C)$ — ангиллын урьдчилсан магадлал (Prior probability)
- $P(X|C)$ — нөхцөлт магадлал (Likelihood)
- $P(C|X)$ — ажиглалтын дараах магадлал (Posterior probability)
- $P(X)$ — нийт магадлал (Evidence)

Урьдчилсан магадлал $P(C)$ нь тухайн ангиллын өмнөх мэдээлэлд үндэслэсэн магадлалыг илэрхийлнэ. Өөрөөр хэлбэл нийт өгөгдөл дотор тухайн ангилал ямар давтамжтай тохиолдож байгааг харуулдаг.

Нөхцөлт магадлал $P(X|C)$ нь өгөгдсөн ангиллын үед ажиглагдсан өгөгдөл үүсэх магадлалыг илэрхийлнэ. Энэхүү судалгаанд тухайн и-мэйл дотор агуулагдаж буй үгсийн давтамжийг ашиглан нөхцөлт магадлалыг тооцоолов.

Дараах магадлал $P(C|X)$ нь ажиглагдсан текстэн өгөгдөл дээр үндэслэн тухайн и-мэйл спам эсвэл хэвийн ангилалд хамаарах магадлалыг илэрхийлнэ.

2.2 Загварын үндсэн таамаглал

Наив Байесын загвар нь тайлбарлагч хувьсагчид ангилал өгөгдсөн нөхцөлд хоорондоо статистикийн хувьд хамааралгүй гэж үздэг.

Үүнийг дараах томъёогоор илэрхийлнэ.

$$P(X_1, X_2, \dots, X_n | C) = \prod_{i=1}^n P(X_i | C)$$

Энд:

- $X_1, X_2, \dots, X_n$ нь текстэн өгөгдөл дэх үгс
- C нь ангилал

Энэхүү таамаглал нь тооцооллыг хялбарчилж, өндөр хэмжээст текстэн өгөгдөл дээр үр ашигтай ангилал хийх боломжийг бүрдүүлдэг.

3 Өгөгдлийн статистик боловсруулалт

3.1 Текст өгөгдлийн боловсруулалт

Текст өгөгдлийг жижиг үсэгт хөрвүүлж, тэмдэгтүүдийг арилган үг болгон задлав.

```
from src.tokenizer import tokenize

sample_email = train_emails[0]

tokens = tokenize(
    sample_email
)

print(
    tokens[:20]
)
```

['subject', 'what', 'up', 'your', 'cam', 'babe', 'what', 'are', 'you', 'looking', 'for', 'if', 'your', '']

3.2 И-мэйлийн уртын статистик

```
email_lengths = [
    len(
        tokenize(email)
    )
    for email in train_emails
]

mean_length = sum(
    email_lengths
) / len(
    email_lengths
)

print(
    "Дундаж урт:",
    round(
        mean_length,
        2
    )
)
```

Дундаж урт: 160.77

3.3 Үгийн сан байгуулах

Текстэн өгөгдлийг статистик загварт ашиглахын өмнө өгөгдөлд агуулагдаж буй бүх давтагдашгүй үгсийг ялган авч үгийн сан (Vocabulary) байгуулсан.

Үгийн сан нь текстэн өгөгдлийг тоон хэлбэрт шилжүүлэх үндсэн суурь болох бөгөөд цаашид ажиглалт бүрийг ижил хэмжээс бүхий вектор орон зайд дүрслэх боломжийг бүрдүүлдэг.

Хэрэв үгийн сангийн хэмжээг V гэж тэмдэглэвэл, ажиглалт бүр V хэмжээст статистик вектороор илэрхийлэгдэнэ.

Өөрөөр хэлбэл:

$$V = |\{w_1, w_2, \dots, w_n\}|$$

энд:

- V нь үгийн сангийн нийт хэмжээ
- w_i нь текст өгөгдөлд ажиглагдсан үг

Энэхүү судалгаанд сургалтын өгөгдлөөс давтагдашгүй үгсийг ялган авч дараах байдлаар үгийн сан байгуулсан.


```

from src.features import build_vocabulary

vocab, word_to_index = \
    build_vocabulary(
        train_emails
    )

print(
    "Үгийн сангийн хэмжээ:",
    len(vocab)
)

```

Үгийн сангийн хэмжээ: 27865

Дээрх тооцооллын үр дүнд үүссэн үгийн сангийн хэмжээ нь загварын тайлбарлагч хувьсагчдын тоог тодорхойлж байгаа бөгөөд энэ нь магадлалын загварын параметрийн хэмжээг илэрхийлнэ.

```

conclusion.append(
    f"Сургалтын өгөгдлөөс {len(vocab)} ширхэг өвөрмөц үг илрүүлэв."
)

```

3.4 Bag-of-Words статистик дүрслэл

Текстэн өгөгдлийг магадлалын ангиллын загварт ашиглахын тулд үгсийн дарааллыг тоон төлөөлөл болгон хувиргах шаардлагатай.

Энэхүү судалгаанд Bag-of-Words аргыг ашиглан ажиглалт бүрийг үгсийн давтамжид суурилсан олон хэмжээст статистик вектор болгон дүрслэв.

Хэрэв V нь үгийн сангийн хэмжээ бол ажиглалт бүр дараах байдлаар илэрхийлэгдэнэ.

$$X = (x_1, x_2, \dots, x_V)$$

Энд:

- X нь тухайн и-мэйлийн статистик вектор
- x_i нь i -р үгийн давтамж
- V нь үгийн сангийн нийт хэмжээ

Ингэснээр текстэн өгөгдөл нь машин сургалтын алгоритмд ашиглагдах тоон шинж тэмдгийн орон зайд шилжиж байна.

Өөрөөр хэлбэл ажиглалт бүр дараах матрицын нэг мөр болно.

$$X \in \mathbb{R}^{n \times V}$$

Энд:

- n нь ажиглалтын тоо
- V нь шинж тэмдгийн тоо

Дээрх статистик дүрслэлийг дараах байдлаар тооцоолов.

```
from src.features import create_bow_matrix

bow_matrix = create_bow_matrix(
    train_emails,
    word_to_index
)

print(
    "Ажиглалтын тоо:",
    len(bow_matrix)
)

print(
    "Нэг ажиглалтын хэмжээс:",
    len(bow_matrix[0])
)
```

Ажиглалтын тоо: 2000

Нэг ажиглалтын хэмжээс: 27865

Үүссэн матрицын мөр бүр нэг и-мэйлийг, багана бүр нэг үгийг төлөөлж байна.

Ийм байдлаар текстэн өгөгдлийг магадлалын статистик шинжилгээнд ашиглах боломж бүрдэж байна. #
Наив Байесын загварын параметрийн үнэлгээ ## Ангиллын шийдвэрийн дүрэм

Энэхүү судалгаанд текстэн өгөгдлийг магадлалын үндсэн дээр ангилах Наив Байесын статистик загварыг ашиглав.

Уг загвар нь ажиглагдсан өгөгдөл X өгөгдсөн үед тухайн объект тодорхой ангилалд хамаарах дараах магадлалыг тооцоолдог.

$$P(C | X) = \frac{P(X | C)P(C)}{P(X)}$$

Энд:

- C нь ангилал
- X нь ажиглагдсан өгөгдөл
- $P(C)$ нь урьдчилсан магадлал
- $P(X | C)$ нь нөхцөлт магадлал
- $P(C | X)$ нь дараах магадлал

Ангиллын шийдвэрийг хамгийн өндөр магадлалтай ангиллыг сонгох зарчмаар гаргана.

$$\hat{C} = \arg \max_C P(C | X)$$

Энд $\hat{C}$ нь таамаглагдсан ангилал болно.

3.5 Урьдчилсан магадлалын үнэлгээ

Сургалтын өгөгдөлд тулгуурлан ангиллын урьдчилсан магадлалыг дараах байдлаар үнэлэв.

$$P(C) = \frac{N_C}{N}$$

Энд:

- N_C нь тухайн ангиллын ажиглалтын тоо
- N нь нийт ажиглалтын тоо

```
spam_count = sum(  
    label == "spam"  
    for label in train_labels  
)  
  
ham_count = sum(  
    label == "ham"  
    for label in train_labels  
)  
  
total_count = len(  
    train_labels  
)  
  
prior_spam = (  
    spam_count /  
    total_count  
)  
  
prior_ham = (  
    ham_count /  
    total_count  
)  
  
print(  
    "Spam prior:",  
    round(  
        prior_spam,  
        4  
    )  
)  
  
print(  
    "Ham prior:",  
    round(  
        prior_ham,  
        4  
    )  
)
```

Spam prior: 0.5

Ham prior: 0.5

3.6 Нөхцөлт магадлалын үнэлгээ

Текстэн өгөгдөлд өмнө нь ажиглагдаагүй үгсээс шалтгаалан магадлал тэг болох асуудлыг шийдвэрлэхийн тулд Лапласын smoothing ашиглав.

$$P(w | C) = \frac{\text{count}(w, C) + \alpha}{N_C + \alpha V}$$

Энд:

- $\text{count}(w, C)$ нь үгийн давтамж
- N_C нь тухайн ангиллын нийт үгийн тоо
- V нь үгийн сангийн хэмжээ
- α нь smoothing параметр

Энэхүү арга нь магадлалын үнэлгээг илүү тогтвортой болгодог.

3.7 Загвар сургах

Дээрх статистик параметруудийг сургалтын өгөгдөл дээр дараах байдлаар үнэлэв.

```
from src.naive_bayes import NaiveBayes

model = NaiveBayes(
    alpha=1
)

model.fit(
    train_emails,
    train_labels
)
```

Сургалтын үр дүнд ангиллын урьдчилсан магадлал, үгийн нөхцөлт магадлал, болон шийдвэрийн дүрэм тооцоологдов. # Загварын үнэлгээний статистик үзүүлэлтүүд

Боловсруулсан ангиллын загварын үр ашиг болон статистик гүйцэтгэлийг хөгжүүлэлтийн өгөгдөл дээр үнэлэв.

Энэхүү судалгаанд ангиллын чанарыг үнэлэх зорилгоор confusion matrix, accuracy, precision, recall болон F1-score үзүүлэлтүүдийг ашиглав.

3.8 Confusion matrix

Confusion matrix нь ангиллын үр дүнг зөв болон алдаатай ангилсан тохиолдлуудаар харуулдаг статистик хүснэгт юм.

$$M = \begin{bmatrix} TP & FP \\ FN & TN \end{bmatrix}$$

Энд:

- TP — спам өгөгдлийг зөв ангилсан тоо
- TN — хэвийн өгөгдлийг зөв ангилсан тоо
- FP — хэвийн өгөгдлийг спам гэж буруу ангилсан тоо
- FN — спам өгөгдлийг хэвийн гэж буруу ангилсан тоо

Хөгжүүлэлтийн өгөгдөл дээр дараах байдлаар үнэлгээ хийв.

```
from src.metrics import (
    accuracy_score,
    confusion_matrix
)

predictions = model.predict(
    dev_emails
)

accuracy = accuracy_score(
    predictions,
    dev_labels
)

tp, tn, fp, fn = \
    confusion_matrix(
        predictions,
        dev_labels
    )

print(
    "TP:",
    tp
)

print(
    "TN:",
    tn
)

print(
    "FP:",
    fp
)
```

```
print(
    "FN:",
    fn
)
```

TP: 491
 TN: 481
 FP: 19
 FN: 9

3.9 Accuracy

Accuracy нь нийт ажиглалтаас зөв ангилагдсан хувь хэмжээг илэрхийлнэ.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

```
print(
    "Accuracy:",
    round(
        accuracy*100,
        2
    ),
    "%"
)
```

Accuracy: 97.2 %

3.10 Precision

Precision нь спам гэж ангилагдсан ажиглалтуудаас үнэхээр спам байсан ажиглалтын хувийг илэрхийлнэ.

$$Precision = \frac{TP}{TP + FP}$$

3.11 Recall

Recall нь бодит спам ажиглалтуудаас хэдийг зөв илрүүлснийг илэрхийлнэ.

$$Recall = \frac{TP}{TP + FN}$$

3.12 F1-score

F1-score нь precision болон recall үзүүлэлтүүдийн зохицуулсан дундаж юм.

$$F1 = \frac{2PR}{P + R}$$

Энд:

- P нь precision
- R нь recall

Дээрх үзүүлэлтүүдийг дараах байдлаар тооцоолов.

```
precision = (  
    tp /  
    (  
        tp + fp  
    )  
)  
  
recall = (  
    tp /  
    (  
        tp + fn  
    )  
)  
  
f1 = (  
    2 *  
    precision *  
    recall  
    /  
    (  
        precision +  
        recall  
    )  
)  
  
print(  
    "Precision:",  
    round(  
        precision*100,  
        2  
    ),  
    "%"  
)  
  
print(  
    "Recall:",  
    round(  
        recall*100,  
        2  
    ),  
    "%"
```

```

        2
    ),
    "%")
)

print(
    "F1:",
    round(
        f1*100,
        2
    ),
    "%")
)

```

Precision: 96.27 %

Recall: 98.2 %

F1: 97.23 %

3.13 Confusion matrix дүрслэл

```

matrix = [
    [tp, fn],
    [fp, tn]
]

plt.figure(
    figsize=(4,3)
)

plt.imshow(
    matrix
)

plt.xticks(
    [0,1],
    ["Spam", "Ham"]
)

plt.yticks(
    [0,1],
    ["Spam", "Ham"]
)

plt.xlabel(
    "Predicted"
)

plt.ylabel(
    "Actual"
)

```



```

)

for i in range(2):
    for j in range(2):
        plt.text(
            j,
            i,
            matrix[i][j]
        )

plt.colorbar()

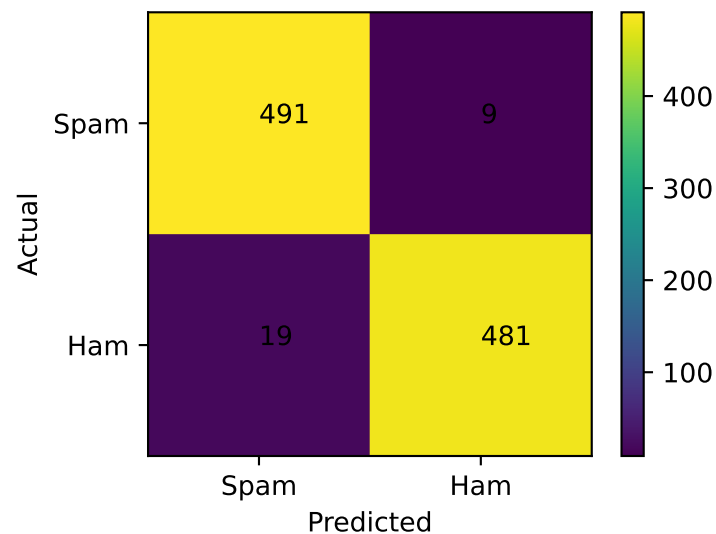
plt.show()

print(
    "Matrix total:",
    tp + tn + fp + fn
)

print(
    "Dev total:",
    len(dev_labels)
)

```

Figure 2: Ангиллын confusion matrix



Matrix total: 1000

Dev total: 1000

4 Үр дүнгийн статистик тайлбар

Хөгжүүлэлтийн өгөгдөл дээр хийсэн туршилтын үр дүнд Наив Байесын ангиллын загвар өндөр статистик гүйцэтгэлтэй ажиллаж байгааг тогтоов.

Туршилтын үр дүнгээс:

```
print(
    f"Accuracy = {accuracy*100:.2f}%"
)

print(
    f"True Positive = {tp}"
)

print(
    f"True Negative = {tn}"
)

print(
    f"False Positive = {fp}"
)

print(
    f"False Negative = {fn}"
)
```

Accuracy = 97.20%
True Positive = 491
True Negative = 481
False Positive = 19
False Negative = 9

үзүүлэлтүүд гарсан.

```
conclusion.append(
    f"Загвар нийт ажиглалтуудын {accuracy*100:.2f} хувийг зөв ангилсан."
)

conclusion.append(
    f"Precision үзүүлэлт {precision*100:.2f} хувь гарсан."
)

conclusion.append(
    f"Recall үзүүлэлт {recall*100:.2f} хувь гарсан."
)

conclusion.append(
    f"F1-score үзүүлэлт {f1*100:.2f} хувь гарсан."
)
```

Accuracy үзүүлэлт 97.20 хувь байгаа нь нийт ажиглалтуудын дийлэнх хэсгийг зөв ангилсныг харуулж байна.

Precision өндөр байгаа нь спам гэж ангилагдсан ажиглалтуудын ихэнх нь үнэхээр спам байсныг илэрхийлж байна.

Recall өндөр байгаа нь бодит спам мэдээллүүдийг алдах магадлал бага байгааг харуулж байна.

F1-score өндөр байгаа нь precision болон recall үзүүлэлтүүдийн хооронд сайн тэнцвэр хадгалагдаж байгааг илэрхийлж байна.

Иймээс боловсруулсан магадлалын загвар нь текстэн өгөгдөл дээр өндөр үр ашигтай ажиллаж байгааг статистикийн үр дүнгээр баталж байна.

5 Загварын хязгаарлалт

Хэдийгээр боловсруулсан загвар өндөр нарийвчлалтай ажилласан боловч дараах статистик хязгаарлалтууд оршиж байна.

Нэгдүгээрт, Наив Байесын загвар нь тайлбарлагч хувьсагчид ангилал өгөгдсөн нөхцөлд хоорондоо статистикийн хувьд хараат бус гэж үздэг.

$$P(X_1, X_2, \dots, X_n | C) = \prod_{i=1}^n P(X_i | C)$$

Бодит текстэн өгөгдөлд үгс хоорондоо утгын болон контекстийн холбоотой байх боломжтой тул энэхүү таамаглал бүрэн хангагдахгүй байх боломжтой.

Хоёрдугаарт, Bag-of-Words арга нь үгсийн дараалал, өгүүлбэрийн бүтэц, утгын хамаарлыг тооцохгүй зөвхөн давтамжийг ашигладаг.

Гуравдугаарт, сургалтын өгөгдлийн хэмжээ болон чанар нь магадлалын үнэлгээний нарийвчлалд шууд нөлөөлдөг.

Иймээс цаашдын судалгаанд илүү том хэмжээний өгөгдөл, семантик төлөөлөл, болон гүн сургалтын аргуудтай харьцуулах боломжтой.

6 Дүгнэлт

Энэхүү судалгааны ажлаар спам болон хэвийн и-мэйлийг ангилах магадлалын статистик загварыг боловсруулав.

Судалгааны хүрээнд:

```
from IPython.display import (
    display_latex,
    Latex
)

latex_code = (
    "\\begin{enumerate}\\n"
    +
    "\\n".join(
        f"\\item {text}"
        for text in conclusion
    )
    +

```

```

"\n\\end{enumerate}"
)

display_latex(
    Latex(
        latex_code
    )
)

```

1. Сургалтын өгөгдлөөс 27865 ширхэг өвөрмөц үг илрүүлэв.
2. Загвар нийт ажиглалтуудын 97.20 хувийг зөв ангилсан.
3. Precision үзүүлэлт 96.27 хувь гарсан.
4. Recall үзүүлэлт 98.20 хувь гарсан.
5. F1-score үзүүлэлт 97.23 хувь гарсан.

Дээрх үр дүнгээс Наив Байесын магадлалын ангиллын арга нь өндөр хэмжээст текстэн өгөгдөл дээр өндөр үр ашигтай ажиллах боломжтой болохыг статистикийн шинжилгээгээр тогтоов.

7 Багийн гишүүдийн оролцоо

Энэхүү судалгааны ажлыг багийн гишүүд хамтын ажиллагааны үндсэн дээр гүйцэтгэсэн бөгөөд судалгааны үе шат бүрт дараах байдлаар оролцоо үзүүлэв.

Багийн гишүүн	Гүйцэтгэсэн ажил
Б.Билгүүн	Судалгааны аргачлал боловсруулах, статистик загвар байгуулах, үр дүнг тайлбарлах
Г.Амин-Эрдэнэ	Өгөгдөл цуглуулах, өгөгдөл боловсруулах, текстэн өгөгдлийн шинжилгээ хийх
Д.Жаргал	Туршилтын орчин бэлтгэх, загвар сургах, үнэлгээний үзүүлэлтүүд тооцоолох
М.Булганлхам	Судалгааны баримт бичиг боловсруулах, онолын хэсэг боловсруулах, материал нэгтгэх

Багийн гишүүд судалгааны бүх үе шатанд харилцан санал солилцож, туршилтын үр дүнг хамтран шинжилж, тайланг эцэслэн боловсруулав.

8 Ашигласан материал

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

GeeksforGeeks. (2024, February 27). *Naive Bayes classifiers*. GeeksforGeeks.

<https://www.geeksforgeeks.org/machine-learning/naive-bayes-classifiers/>

Scikit-learn Developers. (n.d.). *Naive Bayes*. Scikit-learn.

https://scikit-learn.org/stable/modules/naive_bayes.html

Analytics Vidhya. (n.d.). *Naive Bayes classifier explained*. Analytics Vidhya.

<https://www.analyticsvidhya.com/blog/2017/09/naive-bayes-explained/>

Venkatesh. (n.d.). *Spam mails dataset*. Kaggle.

<https://www.kaggle.com/datasets/venky73/spam-mails-dataset>

Python Software Foundation. (n.d.). *Python documentation*. Python.

<https://docs.python.org/3/>

Matplotlib Development Team. (n.d.). *Matplotlib documentation*. Matplotlib.

<https://matplotlib.org/>